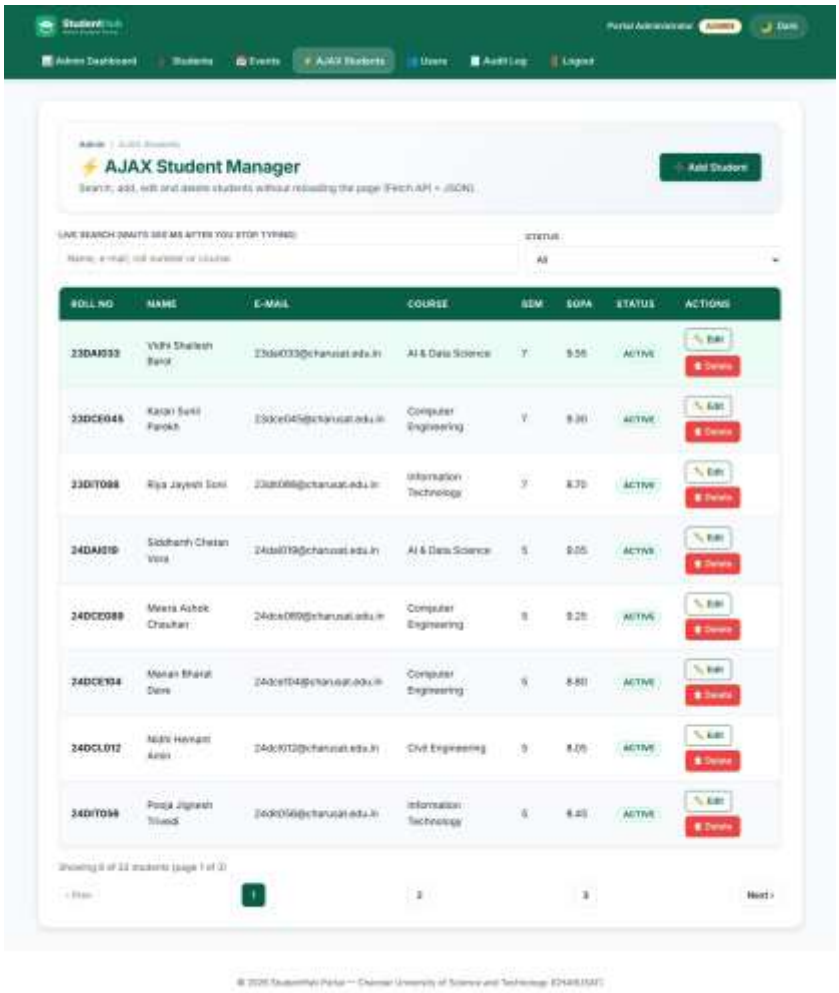
PRACTICAL-13

AIM: Create a PHP JSON API (GET / POST / PUT / DELETE) and use the Fetch API (AJAX) for CRUD without page reload, with loading, success and error states.

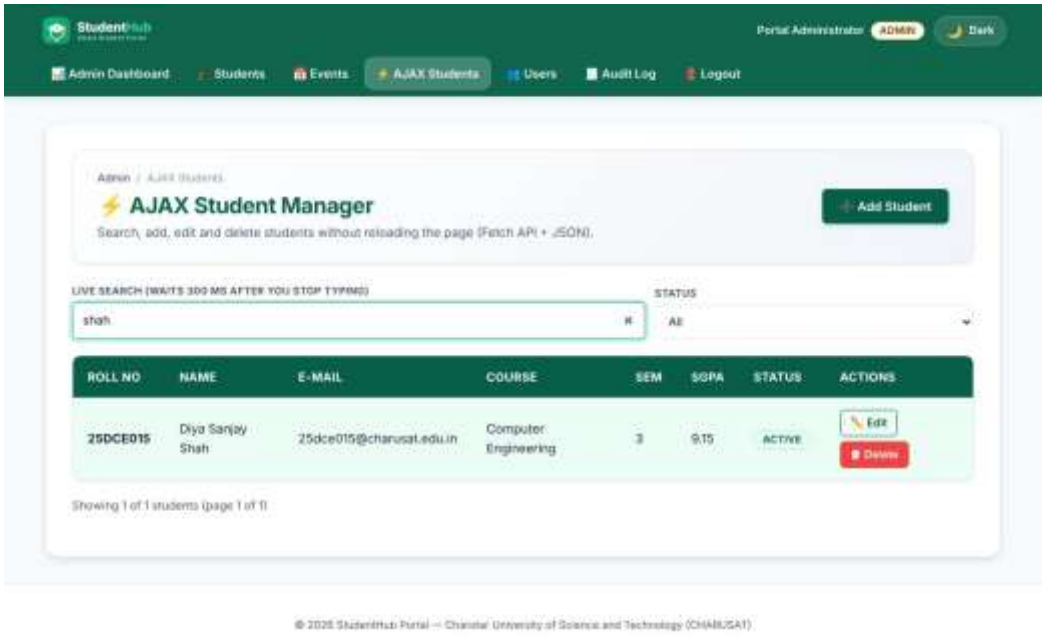
SOURCE CODE :

https://github.com/Aarav280907/WEB/tree/main/Practical(1-4)(Studenthub)/Practical-13

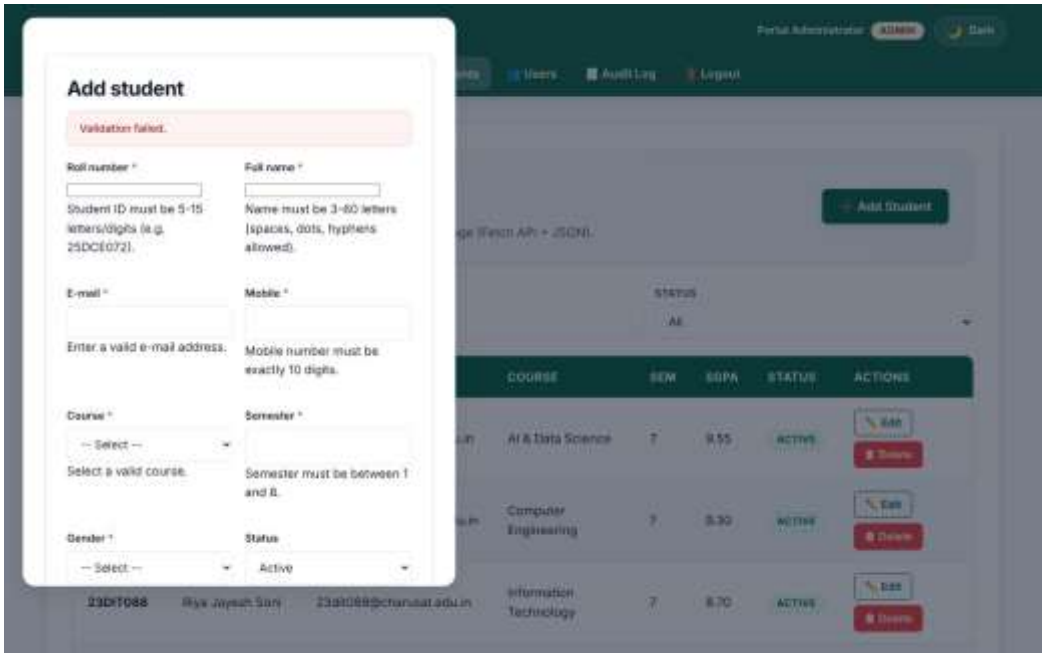
PREVIEW:



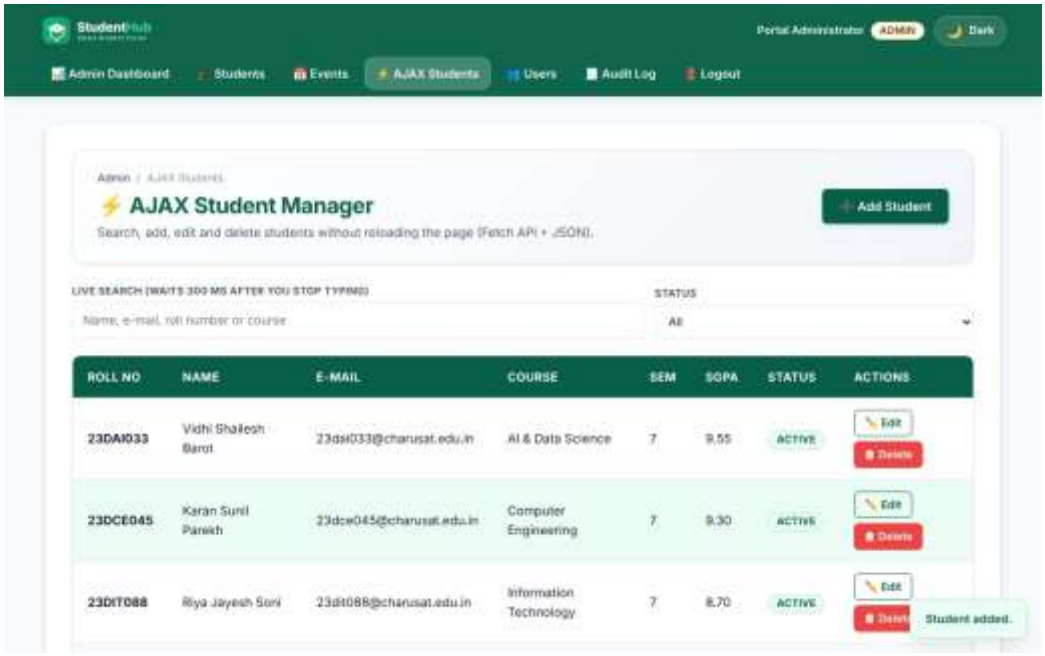
AJAX student manager - rows loaded with fetch() from api/students.php



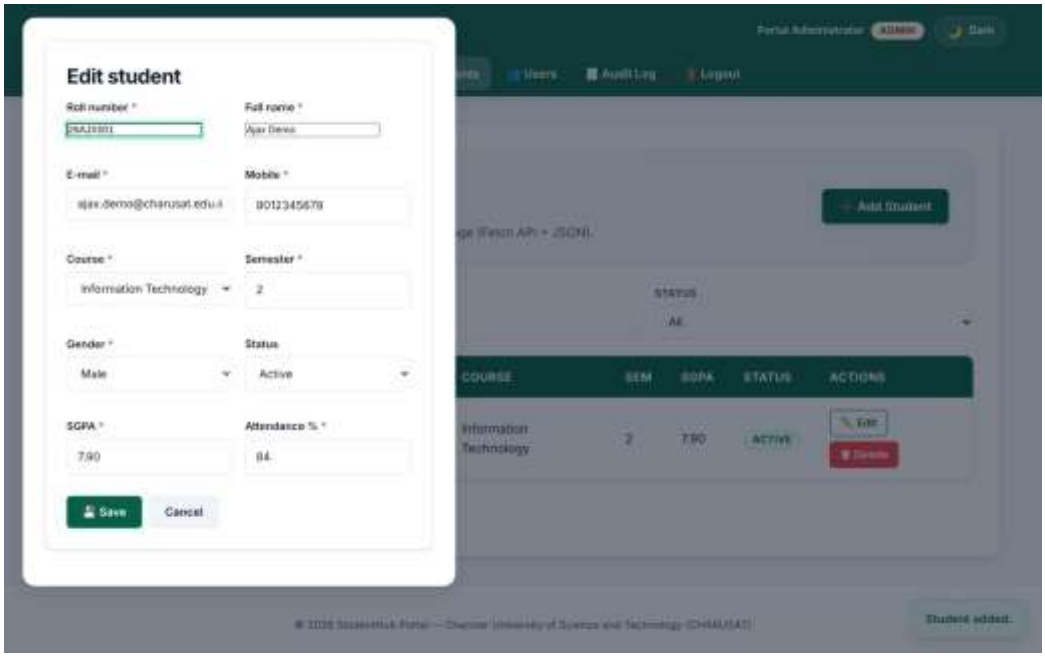
Live search with 300 ms debounce (intermediate extension)



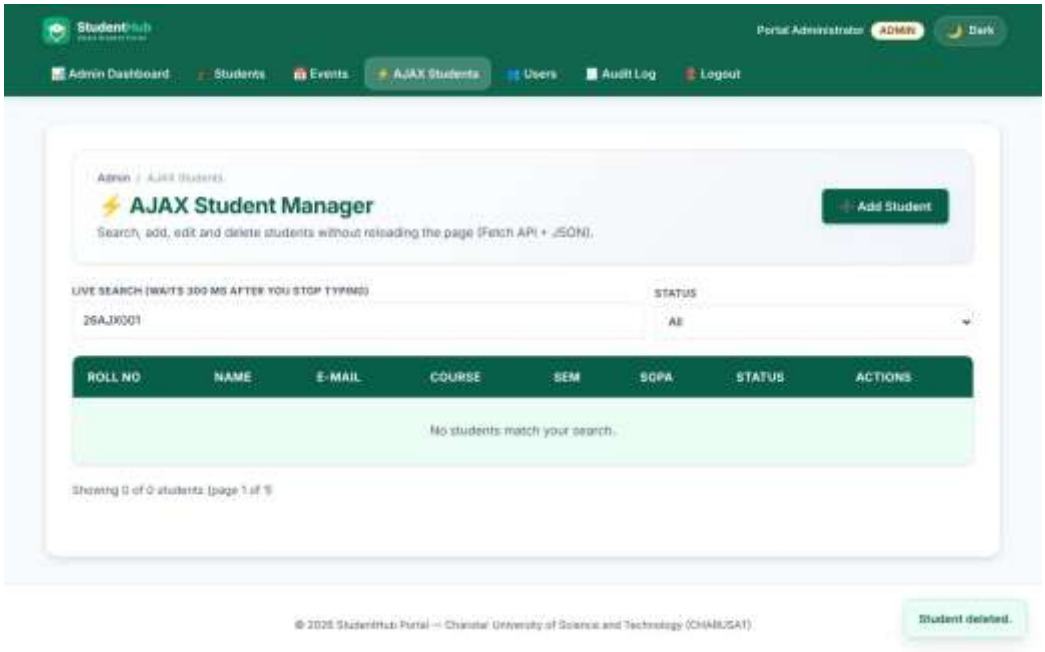
Server validation errors (HTTP 422) shown inside the dialog



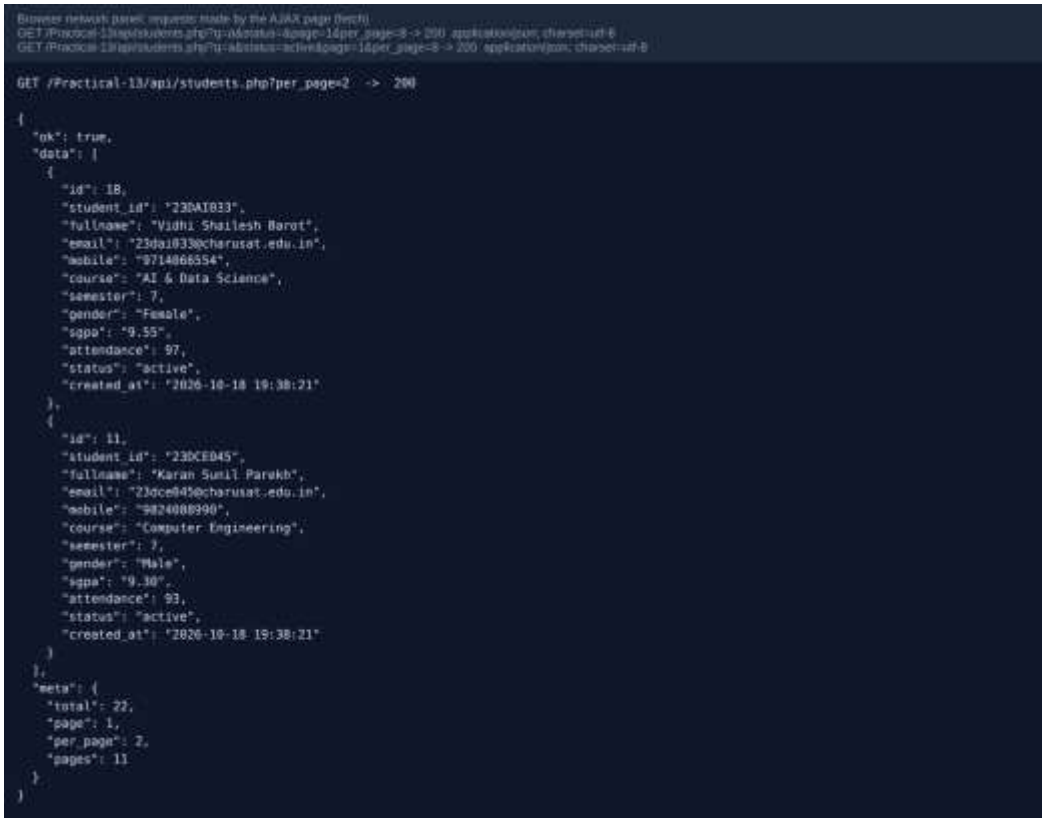
POST - student created, success toast, table refreshed



GET ?id= then PUT - edit dialog filled from JSON



DELETE - row removed without reloading the page



Browser network requests and the JSON response (application/json)



Centralized JSON error responses: 401, 422, 403, 405

TEST CASES :

#	Input / action	Expected result	Result
1	GET api/students.php without login	401 JSON {"ok":false,"error":{...}}	Pass
2	GET as student	403 Administrator access required	Pass
3	GET list as admin	200 JSON with data[] and meta{total,page,per_page,pages}	Pass
4	GET ?q=patel&per_page=3	Filtered, paginated result	Pass
5	GET ?status=zzz	400 Bad Request	Pass
6	GET ?id=99999	404 Not Found	Pass
7	PATCH	405 with Allow header	Pass
8	POST without X-CSRF-Token	403	Pass
9	POST malformed JSON	400	Pass
10	POST invalid fields	422 with a "fields" object	Pass
11	POST valid student	201 Created with the new row	Pass
12	POST duplicate roll number	409 Conflict	Pass
13	PUT ?id=N	200, row updated	Pass
14	DELETE ?id=N, then again	200, then 404	Pass
15	events.php GET (status=open) and POST	List 200; create 201 (admin only)	Pass
16	Server stopped / request blocked	Error banner "Network error - is the server running?"	Pass

KEY QUESTIONS :

1. Do endpoints return valid JSON?

Yes. bootstrap.php sets Content-Type: application/json for every endpoint and all responses go through json_out() / json_error(), which use json_encode. Even uncaught exceptions are turned into a JSON 500 by a central exception handler; the real message is only written to the error log.

```
{"ok":true,"data":[{...}],"meta":{"total":22,"page":1,"per_page":8,"pages":3}}
{"ok":false,"error":{"code":422,"message":"Validation failed.","fields":{"email":"Enter a valid e-mail address."}}}
```

2. Are GET and POST handled?

Yes, and also PUT and DELETE for students. api_methods([...]) allows only the listed methods and answers 405 with an Allow header for others. Writes read a JSON body (php://input) and need the X-CSRF-Token header; reads are open to admins only. events.php supports GET (list/one) and POST (create).

```
$method = api_methods(['GET', 'POST', 'PUT', 'DELETE']);
$admin = api_require_admin();
if ($method !== 'GET') api_csrf();
```

3. Are loading, success and error states shown?

Loading: a status message with aria-live and aria-busy on the table body. Success: a toast ("Student added."). Errors: field messages from the 422 response inside the dialog, a banner for list failures, a toast for edit/delete failures; the Save button is disabled and shows "Saving..." during a request. A request counter ignores stale answers when the user types quickly.

```
async function api(url, opts) {
  const res = await fetch(url, { ...opts, headers, credentials: "same-origin" });
  const body = await res.json();
  if (!res.ok || body.ok === false) throw { status: res.status, message:
body.error.message, fields: body.error.fields };
  return body;
}
```

4. Is input validated before database operations?

Yes. The same validate_student() / validate_event() functions used by the normal forms run on the decoded JSON before any SQL; on errors the API answers 422 and does not touch the database. SQL always uses prepared statements, and a duplicate key (MySQL 1062) becomes a 409 instead of a crash.

5. Advanced extension - status codes and centralized error handling

The API uses 200, 201, 400, 401, 403, 404, 405, 409, 422 and 500 consistently and all of them share one error envelope created in one place (json_error), so the JavaScript has a single way to read errors.